LAB GUIDE · BEGINNER · APPROX. 2 HOURS

ANSIBLE FOR NETWORK AUTOMATION: FROM UBUNTU INSTALL TO YOUR FIRST CHANGE PLAYBOOK

In this lab you turn an Ubuntu workstation into an Ansible control node, connect it to a small set of Cisco IOS devices, and write a playbook that safely creates VLANs, assigns switch ports and updates interface descriptions — with a backup, a dry run and a verification step built in.

YOU WILL

Install Ansible, build an inventory, protect credentials with Vault, and run ad-hoc commands and playbooks.

YOU NEED

Ubuntu 22.04 or 24.04 with sudo, and SSH access to one or more Cisco IOS / IOS-XE devices (physical, CML, GNS3 or EVE-NG).

ASSUMES

Basic Linux shell use and basic Cisco CLI. No prior Ansible or YAML experience.

CONTENTS

1. Lab topology
2. Install Ansible on Ubuntu
3. Create the project and configuration
4. Build the inventory and protect credentials
5. Test connectivity with ad-hoc commands
6. Your first playbook: read-only
7. Change playbook: VLANs, ports, descriptions
8. Dry run, apply and verify
9. Troubleshooting
10. Command reference

01 LAB TOPOLOGY

The control node runs Ansible and reaches every device over SSH on the management network. Nothing is installed on the network devices — Ansible is **agentless** and talks to them through their normal CLI.

HOST	ROLE	PLATFORM	MGMT IP
ctrl	Ansible control node	Ubuntu 24.04 LTS	192.168.10.5
r1	Edge router	Cisco IOS / IOS-XE	192.168.10.11
r2	Edge router	Cisco IOS / IOS-XE	192.168.10.12
sw1	Access switch	Cisco IOS (L2)	192.168.10.21

Table 1 — Lab hosts. Substitute your own addresses throughout the guide.

Each device needs SSH enabled, a local user with privilege access (or an enable secret), and a route back to the control node. A minimal device-side setup looks like this:

```
hostname sw1
ip domain-name lab.local
crypto key generate rsa modulus 2048
username admin privilege 15 secret LabPass123!
enable secret LabEnable123!
ip ssh version 2
line vty 0 4
 login local
 transport input ssh
```

Listing 1 — Device prerequisites (Cisco IOS CLI, config mode).

02 INSTALL ANSIBLE ON UBUNTU

Ubuntu's own repositories carry Ansible, but it is often several releases behind. The official Ansible PPA gives you a current version that Ubuntu still manages through `apt`.

STEP 2.1 — UPDATE THE SYSTEM AND ADD THE PPA

```
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install -y software-properties-common
$ sudo add-apt-repository --yes --update ppa:ansible/ansible
$ sudo apt install -y ansible python3-paramiko
```

Listing 2 — Installing Ansible from the official PPA. `python3-paramiko` provides the SSH library network modules use.

STEP 2.2 — VERIFY THE INSTALL

```
$ ansible --version
ansible [core 2.17.x]
  config file = None
  python version = 3.12.x
  jinja version = 3.1.x
```

Listing 3 — Expected output. Exact versions will differ; any core 2.15 or later works for this lab.

STEP 2.3 — INSTALL THE NETWORK COLLECTIONS

Device support ships in **collections**. The PPA package bundles many of them, but install the two this lab relies on explicitly so you get current versions:

```
$ ansible-galaxy collection install cisco.ios ansible.netcommon
$ ansible-galaxy collection list | grep -E "cisco.ios|netcommon"
```

Listing 4 — Installing and confirming collections. Other vendors follow the same pattern, e.g. `arista.eos` or `junipernetworks.junos`.

03 CREATE THE PROJECT AND CONFIGURATION

Keep every lab file in one directory. Ansible looks for an `ansible.cfg` in the current directory first, so settings stay local to the project.

```
$ mkdir -p ~/netlab/{group_vars/all,backups}
$ cd ~/netlab
```

When you finish the lab, the project will look like this:

```
netlab/
├── ansible.cfg          # project settings
├── inventory.yml        # which devices exist
├── group_vars/
│   └── all/
│       ├── vars.yml     # how to connect
│       └── vault.yml    # encrypted passwords
├── facts.yml            # read-only playbook
├── vlans.yml            # change playbook
└── backups/             # config backups
```

Figure 1 — Project layout.

Create `ansible.cfg`:

```
[defaults]
inventory          = inventory.yml
host_key_checking  = False
gathering          = explicit
stdout_callback    = yaml
forks              = 10

[persistent_connection]
connect_timeout    = 30
command_timeout    = 30
```

Listing 5 — `ansible.cfg`. Disabling host-key checking is acceptable in a lab; in production, pre-load known hosts instead.

04 BUILD THE INVENTORY AND PROTECT CREDENTIALS

The **inventory** lists your devices and sorts them into groups. Groups let a playbook target "all switches" instead of naming hosts one by one.

```
all:
  children:
    routers:
      hosts:
        r1:
          ansible_host: 192.168.10.11
        r2:
          ansible_host: 192.168.10.12
    switches:
      hosts:
        sw1:
          ansible_host: 192.168.10.21
```

Listing 6 — `inventory.yml`. YAML uses two-space indentation; never use tabs.

Connection settings shared by every device go in `group_vars/all/vars.yml`. Network devices use the `network_cli` connection rather than Ansible's default Python-over-SSH.

```
ansible_connection: ansible.netcommon.network_cli
ansible_network_os: cisco.ios.ios
ansible_user: admin
ansible_password: "{{ vault_ansible_password }}"
ansible_become: true
ansible_become_method: enable
ansible_become_password: "{{ vault_enable_password }}"
```

Listing 7 — `group_vars/all/vars.yml`. Passwords are references to vault variables, never plain text.

Now create the encrypted vault. Ansible prompts for a vault password, then opens an editor:

```
$ ansible-vault create group_vars/all/vault.yml

# contents typed into the editor:
vault_ansible_password: LabPass123!
vault_enable_password: LabEnable123!
```

Listing 8 — Creating `vault.yml`. Use `ansible-vault edit` to change it later.

CHECK YOUR WORK

`cat group_vars/all/vault.yml` should print a block starting with `$ANSIBLE_VAULT;1.1;AES256`. Then run `ansible-inventory --graph` — you should see *r1* and *r2* under *@routers* and *sw1* under *@switches*.

05 TEST CONNECTIVITY WITH AD-HOC COMMANDS

An **ad-hoc command** runs a single module against a group without writing a playbook — ideal for a first connection test. This asks every device for a `show version`:

```
$ ansible all -m cisco.ios.ios_command \
    -a "commands='show version | include uptime'" --ask-vault-pass

sw1 | SUCCESS => {
    "stdout": ["sw1 uptime is 3 hours, 12 minutes"]
}
r1 | SUCCESS => { ... }
r2 | SUCCESS => { ... }
```

Listing 9 — A successful ad-hoc run. Every host must return SUCCESS before you continue.

If any host fails, jump to **Section 09 — Troubleshooting** before moving on. Most first-run failures are SSH reachability, wrong credentials or a missing enable password.

06 YOUR FIRST PLAYBOOK: READ-ONLY

A **playbook** is a YAML file containing one or more *plays*. Each play targets a group of hosts and runs an ordered list of *tasks*; each task calls one module. Start with a playbook that changes nothing — it only collects facts and prints them.

```
---
- name: Collect basic device facts
  hosts: all

  tasks:
    - name: Gather IOS facts
      cisco.ios.ios_facts:
        gather_subset: min

    - name: Report model and version
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }} is a {{ ansible_net_model }} running {{
ansible_net_version }}"
```

Listing 10 — `facts.yml`.

```
$ ansible-playbook facts.yml --ask-vault-pass
```

Notice the anatomy you will reuse in every playbook: `name` describes intent, `hosts` chooses targets from the inventory, and each task names a fully qualified module such as `cisco.ios.ios_facts`. Values in `{{ }}` are variables filled in per host.

07 CHANGE PLAYBOOK: VLANS, PORTS, DESCRIPTIONS

Now automate real changes on *sw1*. The playbook below follows a safe change pattern you should keep using: **back up → change → verify → save**. It uses Ansible's *resource modules* (`ios_vlans`, `ios_l2_interfaces`, `ios_interfaces`), which describe the *desired state* rather than raw CLI lines — so Ansible only sends commands when something actually differs.

```
---
- name: Standardise access-switch VLANs and ports
  hosts: switches

  vars:
    vlans:
      - { vlan_id: 10, name: USERS }
      - { vlan_id: 20, name: VOICE }
      - { vlan_id: 99, name: MGMT }
    access_ports:
      - { name: GigabitEthernet0/2, vlan: 10, desc: "USERS - desk 2" }
      - { name: GigabitEthernet0/3, vlan: 10, desc: "USERS - desk 3" }
      - { name: GigabitEthernet0/4, vlan: 20, desc: "VOICE - phone 4" }

  tasks:
    - name: 1 · Back up running config
      cisco.ios.ios_config:
        backup: true
        backup_options:
          dir_path: ./backups

    - name: 2 · Ensure VLANs exist
      cisco.ios.ios_vlans:
        config: "{{ vlans }}"
        state: merged

    - name: 3 · Put ports in access mode on the right VLAN
      cisco.ios.ios_l2_interfaces:
        config:
          - name: "{{ item.name }}"
            mode: access
            access:
              vlan: "{{ item.vlan }}"
        state: merged
      loop: "{{ access_ports }}"

    - name: 4 · Set port descriptions and enable ports
      cisco.ios.ios_interfaces:
```

```
      config:
        - name: "{{ item.name }}"
          description: "{{ item.desc }}"
          enabled: true
      state: merged
    loop: "{{ access_ports }}"

  - name: 5 · Verify VLAN table
    cisco.ios.ios_command:
      commands: show vlan brief
    register: vlan_out

  - name: 6 · Show result
    ansible.builtin.debug:
      var: vlan_out.stdout_lines[0]

  - name: 7 · Save config only if something changed
    cisco.ios.ios_config:
      save_when: modified
```

Listing 11 — `vlans.yml`. Edit the port names to match your switch model (e.g. `Ethernet0/2` on virtual IOSvL2).

CONCEPT	WHAT IT DOES IN THIS PLAYBOOK
vars	Keeps the data (VLANs, ports) separate from the logic. To add a VLAN, edit one line — no task changes.
state: merged	Adds or updates what you list and leaves everything else alone. *replaced* and *overridden* are stricter — they remove unlisted settings.
loop / item	Runs the same task once per entry in `access_ports`.
register	Stores a task's output in a variable so later tasks can print or test it.
save_when: modified	Writes to startup-config only when running-config differs, avoiding needless writes.

Table 2 — Key concepts used in `vlans.yml`.

08 DRY RUN, APPLY AND VERIFY

STEP 8.1 — CHECK SYNTAX, THEN DRY-RUN

`--check` reports what *would* change without touching the device; `--diff` shows the exact commands.

```
$ ansible-playbook vlans.yml --syntax-check
$ ansible-playbook vlans.yml --check --diff --ask-vault-pass
```

STEP 8.2 — APPLY THE CHANGE

```
$ ansible-playbook vlans.yml --diff --ask-vault-pass

TASK [2 · Ensure VLANs exist] ************************
changed: [sw1]
commands:
- vlan 10
- name USERS
- vlan 20
- name VOICE
...
PLAY RECAP *******************************************
sw1    : ok=7    changed=4    unreachable=0    failed=0
```

Listing 12 — First run. *changed* counts tasks that modified the device.

STEP 8.3 — PROVE IDEMPOTENCY

Run the exact same command again. Because the device already matches the desired state, nothing is sent:

```
PLAY RECAP *******************************************
sw1    : ok=7    changed=0    unreachable=0    failed=0
```

Listing 13 — Second run. *changed=0* is the goal: safe to re-run any time.

> **VERIFY ON THE DEVICE**
>
> SSH to *sw1* and run `show vlan brief` and `show interfaces status`. VLANs 10, 20 and 99 should exist, and Gi0/2–0/4 should carry the new descriptions. A timestamped backup file should now be in `./backups`.

CHALLENGE EXERCISES

1. Add VLAN 30 named *PRINTERS* and assign it to Gi0/5 by editing only the `vars` block.
2. Write a second play in the same file targeting *routers* that sets a login banner with `cisco.ios.ios_banner`.
3. Move the `vlans` and `access_ports` data into `host_vars/sw1.yml` so each switch can have its own port map.
4. Change `state: merged` to `state: overridden` on the VLAN task and run with `--check --diff`. Explain what it would delete — and why you would be careful with it.

09 TROUBLESHOOTING

Add `-vvv` to any command for detailed connection logs.

SYMPTOM	LIKELY CAUSE	FIX
UNREACHABLE ... timed out	No IP path or SSH disabled	`ping` and `ssh admin@IP` from the control node; check `transport input ssh`.
Authentication failed	Wrong vault values	`ansible-vault view group_vars/all/vault.yml` and correct them.
Command authorization failed / % Invalid input	Not in enable mode	Confirm `ansible_become: true` and the enable password.
network os None is not supported	Group vars not loaded	Check the folder is `group_vars/all/` next to the inventory.
couldn't resolve module cisco.ios...	Collection missing	Re-run `ansible-galaxy collection install cisco.ios`.
Unable to negotiate ... key exchange	Old device ciphers	Upgrade the device image, or install `ansible-pylibssh` and set `ansible_network_cli_ssh_type: libssh`.
Syntax error ... mapping values not allowed	YAML indentation	Use spaces, align keys; run `--syntax-check`.

10 COMMAND REFERENCE

COMMAND	PURPOSE
`ansible --version`	Show version and active config file
`ansible-inventory --graph`	Display hosts and groups
`ansible <group> -m <module> -a "…"`	Run a single ad-hoc task
`ansible-playbook x.yml --syntax-check`	Validate YAML and structure
`ansible-playbook x.yml --check --diff`	Dry run showing proposed changes
`ansible-playbook x.yml --limit sw1`	Run against one host only
`ansible-vault create | edit | view`	Manage encrypted variable files
`ansible-doc cisco.ios.ios_vlans`	Read a module's options and examples offline

Next steps. Put the *netlab* folder under Git so every change is reviewed and reversible, then explore roles, Jinja2 templates for full configs, and the `ios_facts` network resources for building compliance reports.